

OS_2019-01-28

Disclaimer

Kann Fehler enthalten! Falls du etwas findest bitte auf [Discord](#) bescheid geben!

1) Synchronisation mit Semaphoren (30)

Mit Hilfe der Prozesse *Schiff1*, *Schiff2* und *Passagier* soll der Betrieb von zwei Rundfahrtschiffen simuliert werden.

- *Schiff1*, *Schiff2*: Jeder dieser Prozesse wird genau einmal gestartet und simuliert eines der Schiffe. Die Schiffe fahren abwechselnd. Für jede Rundfahrt lassen sie Passagiere an der Anlegestelle einsteigen, fahren eine Runde und lassen dann die Passagiere wieder aussteigen.
Der Prozess *SchiffX* ($X \in \{1, 2\}$) simuliert das Öffnen und Schließen des Schiffszugangs zum Ein- und Aussteigen an der Anlegestelle durch Aufrufe der Funktionen *zugang_auf()* bzw. *zugang_zu()*. Die Rundfahrt erfolgt durch Aufruf der Funktion *rundfahrt()*.
- *Passagier*: Von diesem Prozess kann es beliebig viele Instanzen im System geben. Er simuliert einen Passagier bei der Schifffahrt.
Die Funktionen *einsteigen()* und *aussteigen()* simulieren das Einsteigen in bzw. Aussteigen aus dem gerade verfügbaren Schiff.

Ergänzen Sie die auf der folgenden Seite gegebenen Prozesstemplates ausschließlich mit **Semaphoroperationen** (die Verwendung von globalen Variablen ist nicht erlaubt), um die Prozesse zu synchronisieren. Lokale Variable und Schleifen dürfen Sie verwenden, wenn Sie diese für Ihre Lösung benötigen. Weiters sind folgende Punkte zu beachten:

- Jedes Schiff kann N Fahrgäste aufnehmen. Das Schiff fährt vom Anlegeplatz ab, wenn es voll ist. Das Einsteigen von Passagieren beginnt, sobald alle Passagiere der vorigen Rundfahrt das Schiff verlassen haben und das andere Schiff voll mit Fahrgästen beladen ist.
- Beim Ein- und Aussteigen kann der Zugang zum Schiff immer nur von einem Passagier gleichzeitig betreten werden.

Initialisierungen

```

init(mutex, 1);
init(board, 0);
init(boarded, 0);
init(leave, 0);
init(left, 0);
init(s1_full, 0); init(s2_full, 1);

```

```
/** Code Schiff1 **/
```

```
for(;;) {
```

```
wait(s2_full);
```

```
zugang_auf();
```

```
for (int i = 0; i < n, i++) {
    signal(board);
    wait(boarded);
}
```

```
zugang_zu();
```

```
signal(s1_full);
```

```
rundfahrt();
```

```
zugang_auf();
```

```
for (int i = 0; i < n, i++) {
    signal(leave);
    wait(left);
}
```

```
zugang_zu();
```

```
}
```

```
/** Code Schiff2 **/
```

```
for(;;) {
```

```
wait(s1_full);
```

```
zugang_auf();
```

```
for (int i = 0; i < n, i++) {
    signal(board);
    wait(boarded);
}
```

```
zugang_zu();
```

```
signal(s2_full);
```

```
rundfahrt();
```

```
zugang_auf();
```

```
for (int i = 0; i < n, i++) {
    signal(leave);
    wait(left);
}
```

```
zugang_zu();
```

```
}
```

```
/** Code Passagier **/
```

```
wait(board);
wait(mutex);
```

```
einsteigen();
```

```
signal(mutex);
signal(boarded);
```

```
/* geniessen */
```

```
wait(leave);
wait(mutex);
```

```
aussteigen();
```

```
signal(mutex);
signal(left);
```

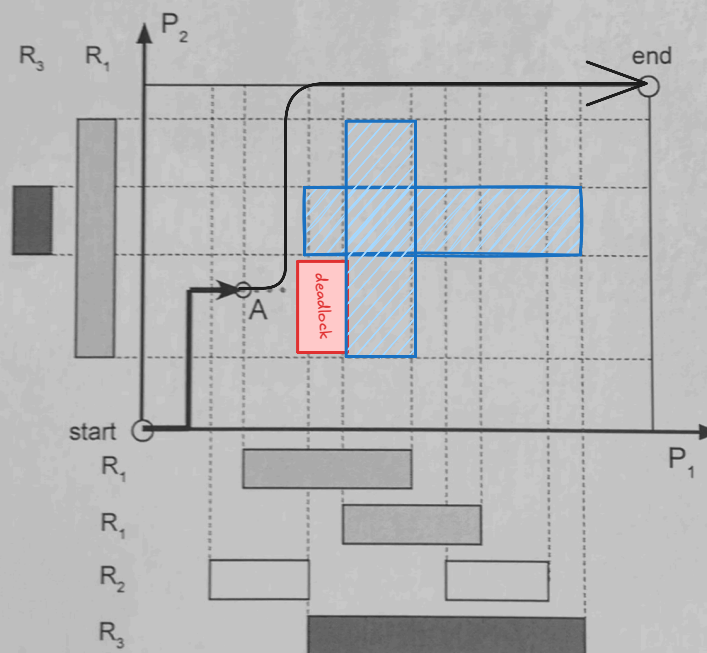
2) Deadlock (30)

Gegeben sind zwei Prozesse, P_1 und P_2 , die Ressourcen der Kategorien R_1 (2 Stück), R_2 (ein Stück) und R_3 (ein Stück) unter Mutual Exclusion verwenden.

Weiters zeigt ein Prozessfortschrittsdiagramm die Ressourcenbelegung von P_1 und P_2 während ihrer Abarbeitung, sowie den Teil einer (quasi)parallelen Abarbeitung der Prozesse als Kantenzug zwischen den Punkten *start* und *A*.

Teilaufgabe A (Prozessfortschrittsdiagramm)

1. Umranden und schraffieren Sie im Fortschrittsdiagramm jene Bereiche, durch die der Kantenzug einer (quasi)parallelen Abarbeitung wegen einer Ressourcenüberschreitung nicht möglich ist.
2. Kennzeichnen Sie auf unterschiedliche Weise die Bereiche, die von einem Kantenzug bei einer deadlockfreien Abarbeitung von P_1 und P_2 nicht passiert werden dürfen. Markieren Sie diesen Bereich deutlich mit dem Wort „Deadlock“.
3. Setzen Sie den gegebenen Kantenzug im Punkt *A* fort, sodass er eine gültige Abarbeitung von P_1 und P_2 darstellt.



Teilaufgabe B (Deadlock Avoidance)

Stellen Sie mit Hilfe des *Banker's Algorithm* fest, ob die durch den Kantenzug gegebene teilweise Abarbeitung der beiden Prozesse in Punkt A durch Fortführung des Prozesses P_1 (d.h., durch die Zuweisung einer Ressource der Kategorie R_1 an P_1) deadlockfrei fortgesetzt werden kann. Erstellen Sie die notwendigen Vektoren und Matrizen und wenden Sie den Banker's Algorithmus an. Beantworten Sie, ob P_1 am Punkt A fortgesetzt werden kann.

Vorbereitung der Vektoren

R gibt an, wie viele Ressourcen es pro Kategorie gibt (2 R1, 1 R2, 1 R3)

V gibt an wie viele Ressourcen gerade in dem Moment wo wir uns befinden noch frei sind (Also hier bei uns bei Punkt A)

$$R = (2, 1, 1)$$

$$V = (1, 0, 1)$$

A gibt an, wie viele von welchem Prozess gerade verwendet werden (P1 verbraucht 0, 1, 0 ...)

$$A = \begin{matrix} P_1 : (0, 1, 0) \\ P_2 : (1, 0, 0) \end{matrix} \implies A = \begin{pmatrix} 0, 1, 0 \\ 1, 0, 0 \end{pmatrix}$$

C gibt an, wie viele Ressourcen ein Prozess maximal anfordert:

$$C = \begin{matrix} P_1 : (2, 1, 1) \\ P_2 : (1, 0, 1) \end{matrix} \implies C = \begin{pmatrix} 2, 1, 1 \\ 1, 0, 1 \end{pmatrix}$$

N ergibt sich durch $C - A$

$$N = C - A = \begin{pmatrix} 2, 0, 1 \\ 0, 0, 1 \end{pmatrix}$$

Berechnung

$R: (2 \ 1 \ 1)$ $V: (1 \ 0 \ 1)$ $Q: (1 \ 0 \ 0)$ Wir werden P1 weitermachen deshalb brauchen wir Ressourcen! $W = V$	$C: \begin{pmatrix} 2 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$	$A: \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$	$N: \begin{pmatrix} 2 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$
$W: (0 \ 0 \ 1)$ $W = W - Q \Rightarrow 001$ $Q_2 = N_2$	$C: \begin{pmatrix} 2 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$	$A: \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$ $A_2: (101) \Rightarrow$	$N: \begin{pmatrix} 1 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$ $N_2: (000)$ $\Rightarrow P_2 \text{ finished} \Rightarrow \text{ress. returned}$ $101 \leq 001$ ✓ $001 \leq 001$ ✓
$W = W + A_2$ $W: (101)$	$C: \begin{pmatrix} 2 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$	$A: \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$	$N: \begin{pmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$ $N_1 \Rightarrow 000$ $\Rightarrow A_1 = (2 \ 1 \ 1)$ $\Rightarrow A_1 = C_1$ $\Rightarrow P_1 \text{ finished}$ $\Rightarrow \text{ress. ret.}$ $101 \leq 101$ ✓

$\Rightarrow$ Safe space

1. Die Ausgangssituation (Die Matrizen)

Zuerst werden die Ressourcen und der aktuelle Zustand definiert:

- **R (Resources):** Gesamtanzahl der Ressourcen im System. Hier: $(2, 1, 1)$.
- **V (Available):** Aktuell verfügbare (freie) Ressourcen. Hier: $(1, 0, 1)$.
- **C (Claim/Max):** Was jeder Prozess maximal benötigen wird.
- **A (Allocation):** Was jeder Prozess aktuell bereits besitzt.
- **N (Need):** Was jeder Prozess noch zusätzlich braucht, um fertig zu werden. Berechnung:
 $N = C - A$.

2. Die Anforderung (Q)

Ein Prozess (hier P_1) stellt eine Anforderung $Q = (1, 0, 0)$.

Bevor das Betriebssystem die Ressourcen zuteilt, wird simuliert, was passieren würde:

1. **Check:** Ist $Q \leq V$? (Haben wir genug?) $\rightarrow (1, 0, 0) \leq (1, 0, 1)$? **Ja**.
2. **Check:** Ist $Q \leq N_1$? (Darf der Prozess das überhaupt noch fordern?) $\rightarrow (1, 0, 0) \leq (2, 0, 1)$? **Ja**.

Simulation der Zuteilung:

- Das neue verfügbare W (Work) wird berechnet: $W = V - Q = (0, 0, 1)$.
- Die Allocation von P_1 erhöht sich: $A_1 = (1, 1, 0)$.
- Der Need von P_1 verringert sich: $N_1 = (1, 0, 1)$.

3. Der Sicherheitsalgorithmus (Safety Check)

Jetzt kommt der Kern der Tabelle: Wir suchen eine Sequenz von Prozessen, die nacheinander fertiggestellt werden können, mit den Ressourcen, die wir gerade "Work" (W) nennen.

Schritt A: Suche einen Prozess, dessen Need $\leq W$ ist.

- Prüfung P_1 : $N_1(1, 0, 1) \leq W(0, 0, 1)$? **Nein**. (Ressource 1 fehlt).

- Prüfung P_2 : $N_2(0, 0, 1) \leq W(0, 0, 1)$? **Ja.**

Schritt B: Prozess P_2 ausführen und Ressourcen zurückgeben.

- Da P_2 fertig werden kann, gibt er nach Abschluss alle seine Ressourcen (A_2) zurück in den Pool.
- Neues $W = W + A_2 = (0, 0, 1) + (1, 0, 0) = (1, 0, 1)$.

Schritt C: Nächsten Prozess prüfen.

- Prüfung P_1 : $N_1(1, 0, 1) \leq W(1, 0, 1)$? **Ja.**
- P_1 kann fertig werden und gibt seine Ressourcen zurück.

4. Ergebnis

Da eine Sequenz gefunden wurde ($P_2 \rightarrow P_1$), in der alle Prozesse beendet werden können, ist der Zustand **sicher (Safe State)**. Die Anforderung Q darf also genehmigt werden.

3) Fragen zu Betriebssystemen (40)

② Nennen sie zwei Abstraktionen, die ein Betriebssystem dem Benutzer bietet. Welche Aufgaben führt das Betriebssystem durch, um diese Abstraktionen zu realisieren? (4)

1. Abstraktion der CPU - Der Prozess:

- Realisierung:
 - CPU-Scheduling
 - Kontext-Wechsel
 - Prozessverwaltung

2. Abstraktion des Speichers durch Dateiensystem

- Realisierung:
 - Space Multiplexing
 - RAM wird aufgeteilt auf mehrere Programme
 - Darf nicht von nicht eigenem Prozess überschreiben
 - Adressübersetzung und Paging
 - logische Adresse in physische Adresse umwandeln
 - Paging zerlegt in gleich große Teile
 - Swapping:
 - Wenn Hauptspeicher voll wird in Sekundärspeicher ausgelagert
 - Dateiverwaltung:
 - Dateinamen etc...

② Wie unterstützt ein Betriebssystem die Portabilität von Software? (2)

Indem beispielsweise IO gegeben ist, mit der man mit der Software interagieren kann und indem das OS eine Abstraktionsschicht der Hardware ist... somit muss man nicht jede Software für jede Hardwarekombination programmieren sondern nur für das Betriebssystem, welches sich dann um die Hardwarezuweisung kümmert. Das Betriebssystem stellt dann mehrere APIs.

② Geben Sie drei Hardwaremechanismen an, mit denen moderne Mikroprozessorsysteme die Aufgaben des Betriebssystems unterstützen. Charakterisieren sie für jeden genannten Mechanismus kurz, wie diese Unterstützung erfolgt. (3)

1. Execution Mode (Privileged Mode vs User Mode):

- Im User mode kann man kritische Bereiche nicht bearbeiten
- OS wird durch direktem Zugriff oder Zerstörung durch Benutzerprogramm geschützt

2. Interrupt Mechanik:

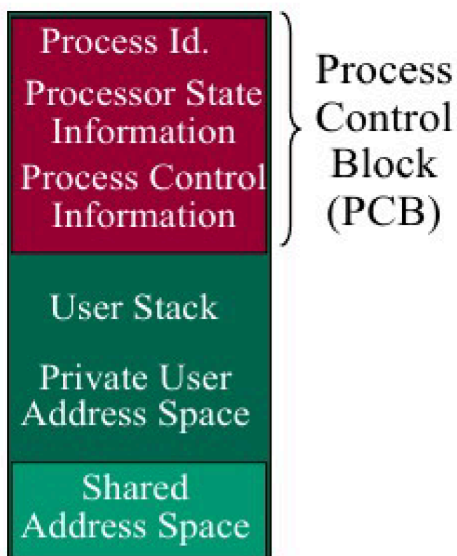
- Durch externe Quelle kann Programmfluss unterbrochen werden
- Signale

- Das unterstützt Multiprogramming und Pre-emptive Scheduling
3. Speichermanagement-Hardware MMU / Adressübersetzung:
- dynamische Übersetzung von logischen zu physischen Adressen durch Translation Lookaside Buffer (TLB) oder Base/Bound Register
 - Unterstützung: das realisiert die Abstraktion vom virtuellen Speichers was man für Relocation braucht (Programme müssen nicht an fixen Adressen liegen...)

② Was versteht man unter einem Process image? Erklären Sie, aus welchen Teilen ein Process image besteht? (4)

Unter einem **Process Image** versteht man die Gesamtheit aller Elemente eines Prozesses (Programmcode, Daten, Stacks und Attribute), die sich im (virtuellen) Speicher befinden. Das Betriebssystem benötigt Teile dieses Images, um den Prozess verwalten zu können.

4. **User Program:** Das eigentliche ausführbare Programm (der Code), das ausgeführt wird.
5. **User Data:** Dies ist der modifizierbare Bereich des User Space. Er beinhaltet die globalen Variablen, Datenpuffer sowie den **User Stack** für die lokale Datenhaltung des Programms.
6. **System Stack (Kernel Stack):** Dieser Stapelspeicher wird verwendet, um Parameter und Rücksprungadressen (Calling Addresses) bei Systemaufrufen (System Calls) oder Prozeduraufrufen des Kernels zu speichern
7. **Process Control Block (PCB):** Dies ist der "Execution Context" des Prozesses. Er enthält alle administrativen Informationen, die das Betriebssystem zur Steuerung benötigt, darunter:



② Nennen Sie zwei verschiedene Möglichkeiten, wie Sie mittels Nachrichtenkommunikation (ohne andere Synchronisationskonstrukte) in einem Computersystem einen konsistenten Datenaustausch zwischen parallelen Prozessen realisieren können. (4)

8. **Synchrone Kommunikation (Rendezvous):** Sender und Empfänger blockieren beide so lange, bis die Nachricht erfolgreich übergeben wurde. Die Synchronisation erfolgt implizit durch das Zusammentreffen beider Prozesse am Übergabepunkt.
9. **Asynchrone Kommunikation (Pufferung):** Nachrichten werden in Mailboxes oder Queues zwischengespeichert. Konsistenz wird durch die **Kopier-Semantik** garantiert: Da der Empfänger eine Kopie erhält und nicht auf den Originalspeicher des Senders zugreift, sind Race Conditions ausgeschlossen.

❓ **Bei welchen der folgenden Scheduling-Strategien kann es zur Starvation kommen:**

- (a) FCFS,
- (b) Shortest Job First,
- (c) Round Robin,
- (d) Priority Scheduling?

Begründen Sie jeweils Ihre Antwort. (4)

Antwort:

- b) weil wenn ein langer Job darauf wartet, dass ein kurzer fertig wird und während er wartet ein weiterer kürzerer kommt, und sich das immer wieder wiederholt, kommt er nie drann
- d) analog zu b) weil es immer einen wichtigeren geben könnte der dazwischengeschoben werden könnte.

❓ **Wir betrachten ein System mit Virtual Memory Management. Diskutieren Sie, welche der**

folgenden Situationen beim Referenzieren einer virtuellen Adresse auftreten bzw. nicht auftreten kann: (4)

- (a) TLB Miss ohne Page Fault
- (b) TLB Miss mit Page Fault
- (c) TLB Hit ohne Page Fault
- (d) TLB Hit mit Page Fault.

Antwort:

(a) TLB Miss ohne Page Fault

• **Status: Möglich (und häufig).**

• **Erklärung:** Dies tritt auf, wenn die gesuchte Seite zwar im Hauptspeicher liegt (daher kein Page Fault), ihr Eintrag aber aktuell nicht im TLB-Cache gespeichert ist.

• **Ablauf:** Der Prozessor findet die Adresse nicht im TLB (Miss). Er muss daraufhin auf die Page Table im Hauptspeicher zugreifen. Da die Seite vorhanden ist (Present Bit gesetzt), wird die physische Adresse ermittelt, der TLB aktualisiert und der Zugriff fortgesetzt, ohne dass Daten von der Festplatte geladen werden müssen.

(b) TLB Miss mit Page Fault

- **Status: Möglich.**
- **Erklärung:** Dies ist der klassische Fall eines Zugriffs auf ausgelagerte Daten.
- **Ablauf:** Die Adresse ist nicht im TLB (Miss). Der Zugriff auf die Page Table zeigt, dass das "Present Bit" nicht gesetzt ist. Das Betriebssystem löst einen Page Fault aus, um die Daten per "Demand Paging" vom Sekundärspeicher (Festplatte) in den Hauptspeicher zu laden. Da die Seite nicht im Speicher war, konnte sie logischerweise auch nicht im TLB (der nur *gültige* Zuordnungen cachen soll) sein.

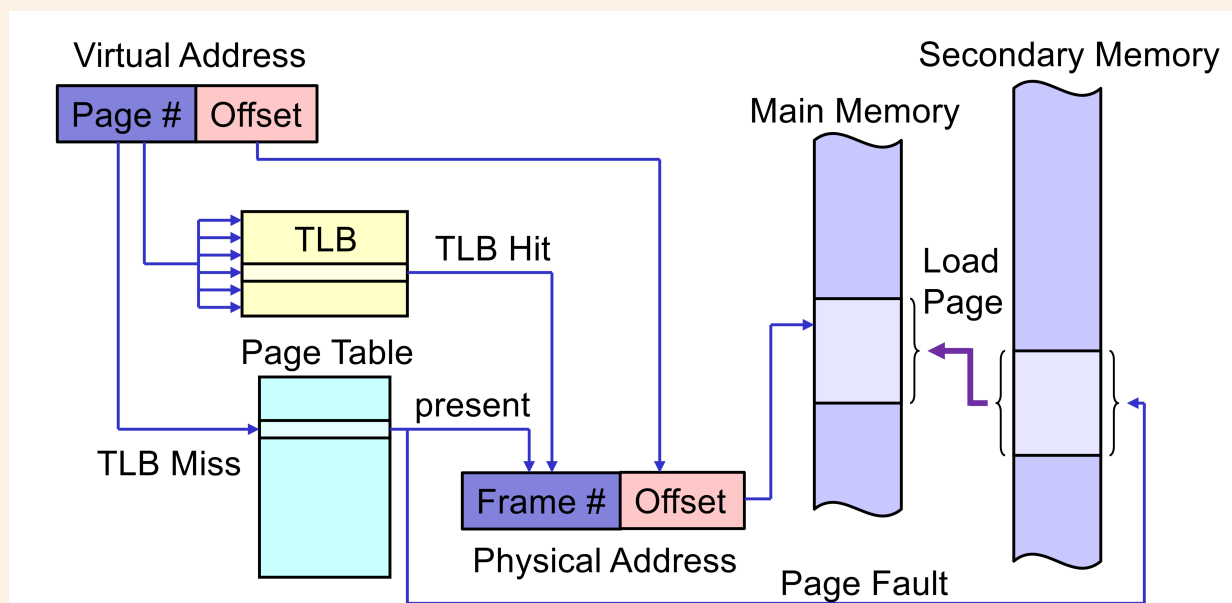
(c) TLB Hit ohne Page Fault

- **Status: Möglich (der Idealfall).**
- **Erklärung:** Dies ist das Zielszenario für eine hohe Systemleistung.
- **Ablauf:** Die Übersetzung der virtuellen Adresse in die physische Adresse (Frame Number) wird direkt im TLB gefunden. Da der Eintrag im TLB existiert, ist die Seite im Hauptspeicher vorhanden. Der Zugriff erfolgt sofort, ohne die Page Table im RAM durchsuchen zu müssen.

(d) TLB Hit mit Page Fault

- **Status: Nicht möglich** (in einem korrekt funktionierenden System).
- **Erklärung:** Ein TLB Hit bedeutet, dass die Memory Management Unit (MMU) eine gültige Zuordnung von Seite zu Frame im Cache gefunden hat und davon ausgeht, dass die Daten im Speicher liegen. Ein Page Fault bedeutet definitionsgemäß, dass die Daten *nicht* im Speicher sind.
- **Widerspruch:** Wenn das Betriebssystem eine Seite auslagert (Swapping), muss es zwingend auch den entsprechenden Eintrag im TLB ungültig machen (invalidieren). Würde ein TLB Hit auftreten, während die Seite fehlt, läge ein Fehler in der Synchronisation zwischen TLB und Page Table vor (Zugriff auf veraltete/falsche Daten). Laut dem Ablaufdiagramm in den Quellen führt ein TLB Hit direkt zur physischen Adresse; der Pfad zum Page Fault wird nur über den "TLB Miss"-Pfad erreicht.

Damit ist das hier gemeint:



man kann nur wenn TLB Miss ist einen Page fault bekommen...

❓ **Beschreiben Sie den Aufbau und die Verwendung einer Multilevel Page Table (eventuell mit Skizze). Warum werden Multilevel Page Tables verwendet? (3)**

Anwendung: große Prozesse, mit großer Page Table, falls Page Table nicht mehr in RAM passt, wird unterteilt in zwei Pagetables (min 3. Zugriffe, dauert länger)

Die Seitentabelle wird selbst aufgeteilt (gepaged) Es gibt einen "Directory" Root-Table der auf Teile der Page Table zeigt.

Die virtuelle Adresse wird nicht einfach nur in eine Seitennummer und ein Offset unterteilt. Stattdessen wird der Teil für die Seitennummer weiter hierarchisch aufgespalten.

1. Outer Page Table:

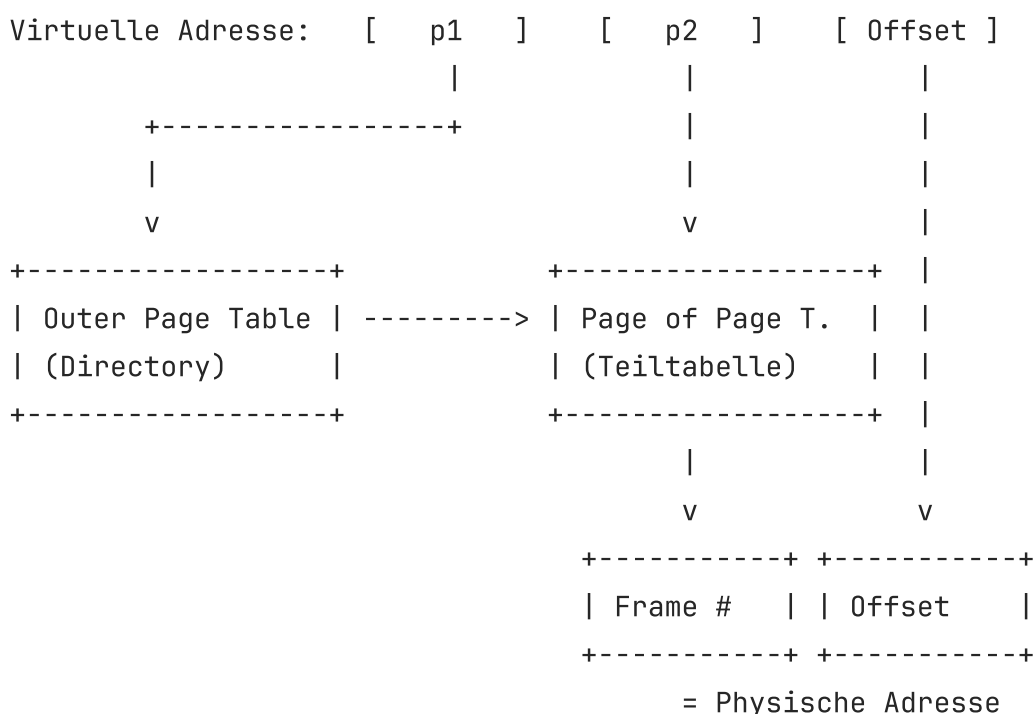
- gilt als Index
- wird typischerweise resident im Speicher gehalten

2. Page of Page Table:

- Verweis aus Root-Table zeigt darauf
- Ist eine Seite der eigentlichen Seitentabelle
- 2. Teil der Adresse wird hier gesucht und zeigt dann zum tatsächlichen Eintrag mit der physischen Frame number.

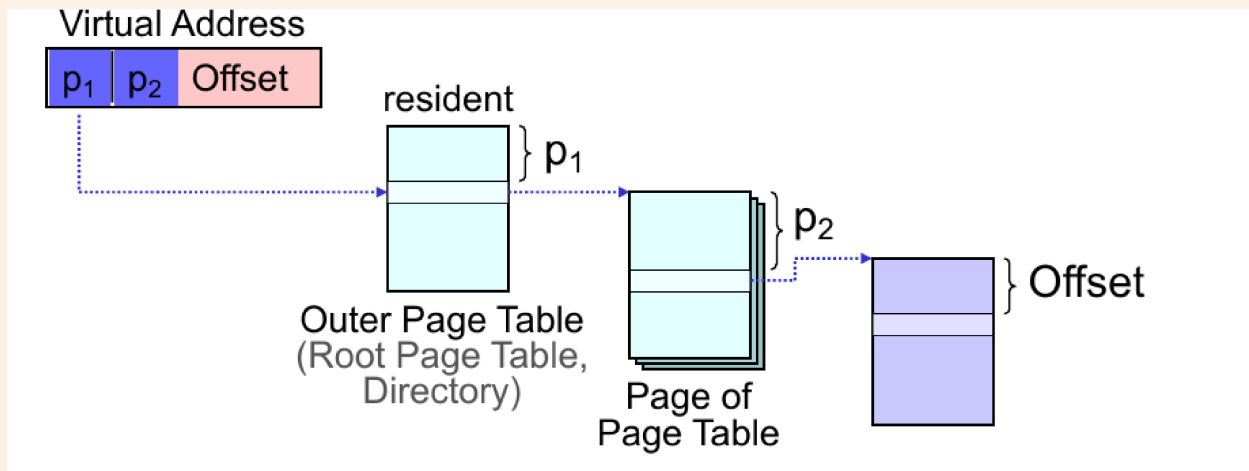
3. Physische Adresse:

- Frame Number die ermittelt wurde, wird mit ursprünglichen Offset kombiniert, damit man auf das Datum im Hauptspeicher zugreifen kann...



Der Hauptgrund für die Verwendung ist die Verwaltung von **großen Prozessen**.

- Wenn ein Prozess einen sehr großen virtuellen Adressraum nutzt, würde eine einfache, einstufige Seitentabelle extrem groß werden.
- Es wäre ineffizient (oder bei Fragmentierung unmöglich), einen so großen zusammenhängenden Speicherbereich nur für die Seitentabelle im RAM freizuhalten.
- Durch die Aufteilung in mehrere Ebenen muss nicht die gesamte Seitentabelle permanent im Speicher sein; Teile der "Page Tables" können ausgelagert werden, solange nur das "Directory" (Root) resident bleibt.



❓ **Welches Dateiformat einer regulären Datei hat für ein Betriebssystem besondere Bedeutung? Warum ist dieses Format wichtig? Wie Wird es erkannt? (2)**

Binary Files: erlauben beliebige Bitmuster und ausführbare Dateien...

Werden mit Magic Number im Header erkannt.

Es ist wichtig, weil das OS muss dieses eigene Exe-File-Format interpretieren können, um das Programm korrekt in den Speicher zu laden und auszuführen. Das Format beinhaltet spezifische Segmente wie den header, den Programmcode (Text), Daten, Relocation Bits und die Symbol Table.

❓ **Mit welcher logischen Struktur des I/O Systems versucht man bei der Realisierung von I/O Funktionen sowohl ein einheitliches Programmierinterface, als auch eine möglichst gerätespezifische Ansteuerung zu erreichen? (4)**

Durch die Schichtung in:

- **Logical I/O** (einheitliches Interface für generelle logische Bedienung vom Gerät (open / close / read...))
- **Device I/O** (Gerätespezifische I/O -> logische Operation wird in eine Sequenz von konkreten I/O Anweisungen übersetzt und gepuffert)
- **Scheduling and Control** (wann wer darf)

❓ **Was versteht man unter Synchronous bzw. Asynchronous I/O? Beschreiben Sie die beiden Arten, I/O-Operationen durchzuführen. (3)**

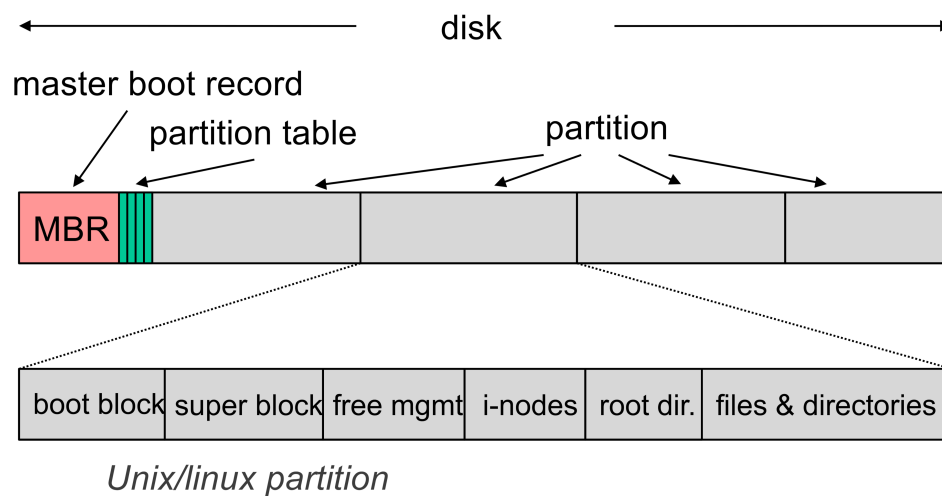
Synchronous:

- der auszuführende Prozess wird blockiert bis die I/O Operation wirklich durchgeführt wurde (Prozess wartet bis Ausgabe am Bildschirm)

Asynchronous:

- Prozess kann die Datei bei einer I/O Operation an einen Buffer weitergeben und parallel zur stattfindenden I/O Operation weiterarbeiten
- Obwohl noch nicht am Bildschirm kann weitergearbeitet werden.

② Beschreiben Sie das typische Layout einer Disk bzw. eines Filesystems. Erklären Sie kurz die einzelnen Komponenten. (3)



Also jede Disk ist aufgeteilt in master boot record mit den partition table und die partitions
jede der partitions hat einen boot block, super block, free mgmt, i-nodes, root dir. und files & directories

- mbr: kümmert sich beim booten darum, dass alles richtig startet indem er die aktive Partition findet und ihrem Boot-Block die Kontrolle gibt
- boot block: der mbr lädt den code aus dem Boot Block in den Speicher und führt ihn aus. Der Code lädt dann den eigentlichen Kernel des OS.
- super block hat die Meta-Daten der Partition
- free management verwaltet die Belegung der Daten
- i-nodes sind die Datenstrukturen für die Metadaten. Jede Datei hat genau einen Inode. Er zeigt auf die Datenblöcke sowie Attribute.
- root-dir ist der Einstiegspunkt in den Verzeichnisbaum.
- files und directories ist das was man darauf speichert.